Introduction:

Overview:

You are to **research a data structure** you have selected from a randomly allocated short list of options; **develop an implementation** and prepare and deliver a **live teaching presentation** to your classmates. Your presentation should explain the structure to an audience at your current level, i.e. those who can program and have studied other data structures but are unfamiliar with your topic.

Requirements:

Your project must address the following areas:

What is the data structure?

- Explain the structure and its underlying principles.
- Define the data structure and explain how it organises and stores information.
- Describe its internal organisation and any important properties or invariants that must be maintained.
- Identify and explain its core abstract data type (ADT) operations.
- Use diagrams or worked examples to illustrate how the structure behaves.

Why do we use it?

- Explain the purpose of the data structure and its strengths and weaknesses.
- Identify the operations or tasks for which it is particularly effective.
- Explain its limitations and the situations in which it performs poorly.
- Discuss the **trade-offs** involved in using this structure.
- Compare it with at least one relevant alternative data structure, explaining when the alternative might be preferable.

When do we use it?

- Explain the circumstances in which the structure is appropriate.
- Identify the types of problems and workloads for which it is well suited.
- Describe at least one realistic application or use case.
- Explain why the structure is appropriate for that application.
- Discuss circumstances in which a different structure would be a better choice.

Implementation and worked examples

- Demonstrate that you understand how the data structure works in practice.
- Implement its core ADT operations.
- Demonstrate the structure in use through a simple, meaningful application or use case.
- Walk through representative operations, explaining the logical steps involved and how the structure changes.
- Explain important implementation decisions and how the code reflects the properties of the structure.
- Demonstrate that the implementation behaves as expected for representative inputs and relevant edge cases.

Remember: A small, correct implementation that you fully understand is preferable to a larger implementation that you cannot explain.

Performance analysis

- Analyse the performance of the data structure.
- Provide the Big-O time complexity of its core operations.
- Explain and justify the complexity of each operation, referring to the structure's properties and your implementation where appropriate.
- Distinguish between best-case, average-case, and worst-case complexity where relevant.
- Discuss any important **space complexity** considerations.
- Relate the performance analysis to the use cases and **trade-offs** discussed above.

Simply listing Big-O values without explaining or justifying them is insufficient.

Note: Favour depth over breadth. The objective is to understand your chosen data structure thoroughly, not to cover every variation or advanced feature.

The data structures you may choose your subject from have been allocated via an **MCQ on Moodle**. Please select your topic of interest by 23:59 on Sunday 15th November.

Deliverables:

Live teaching presentation

You must deliver a live presentation to the class that teaches your allocated data structure. Your presentation must:

- Be approximately **10 minutes** in length.
- Be aimed at an audience on your current level.
- Use clear visual aids, such as diagrams, to explain the structure and its operations.
- Include code to demonstrate the implementation logic.
- Include at least one worked example illustrating how the structure operates.
- Explain the structure's performance using Big-O notation.
- Discuss its strengths, weaknesses, trade-offs, and appropriate applications.
- Demonstrate your own understanding of the material, rather than just reproducing your source information.

You may use slides, diagrams, code demonstrations, or other appropriate teaching materials. The emphasis is on the clarity, accuracy, and depth of your explanation, rather than the visual sophistication of your slides.

Question and answer session

After your presentation, you must participate in a Q&A session lasting approximately 5 minutes. This session forms part of the assessment. Questions may be asked by the lecturer and your classmates. You should be prepared to:

- Clarify points from your presentation.
- Explain the reasoning behind an operation or implementation decision.
- Justify your complexity analysis.
- Reason about the behaviour of the structure in a new or unfamiliar example.
- Discuss the circumstances in which the structure or an alternative would be most appropriate.

You are not expected to know every possible detail about the data structure. However, you should be able to *reason* from the principles you have learned and explain your conclusions.

Associated Git repository

You must submit an associated **Git repository** (**without this, your project will receive 0)** containing:

- An implementation of your data structure's core ADT operations.
- An example demonstrating the structure employed in a simple use case.
- A README containing instructions for running the code and an explanation of how to use the implementation.
- Any relevant tests or examples used to verify the behaviour of the implementation.

Your repository should be organised, understandable, and reproducible. **AI generative tools MAY be used as aids to develop this code base, but you must be able to explain all parts of your code.** Code that you cannot clearly explain will be treated as incorrect and will not be eligible for marks

Code Referencing & Research:

As the focus of this assessment is research and understanding-based, you may consult reliable sources and **adapt** existing code examples, provided all sources are clearly attributed. Include references for the sources used in your research and clearly acknowledge any adapted code, diagrams, or other material.

Generative AI Use:

Generative AI may be used in the development of the associated code base for this project. However, you may not employ generative AI (including explanations or analyses) in the development of your presentation materials.

You must include a **Generative AI Use Declaration** in the README of your Git repository. Failure to provide an accurate declaration may be treated as an academic integrity issue.

If you used generative AI tools at any point during this project, your declaration must:

- Identify the tools used
- Briefly describe how they contributed to your work.
- Specify whether AI was used to generate, modify, debug, *explain*, or test code (identifying the relevant parts of the codebase where applicable)

If you did not use generative AI tools, you must explicitly state this in your declaration.

Whether you employ AI or write your entire submission from scratch, you remain responsible for the correctness and quality of all submitted work. You must understand and be able to explain all code you submit.

Marking Rubric:

		Weight
Conceptual understanding & technical accuracy	Accuracy and depth of the explanation, including the structure's internal organisation, properties, operations, and purpose.	25%
Explanation of operations & worked examples	Clarity and correctness of the worked examples and explanations of the logical steps involved in operating on the structure.	15%
Complexity analysis and critical reasoning	Correctness and justification of Big-O analysis, understanding of relevant trade-offs, and quality of comparisons with alternative structures.	20%
Implementation and supporting evidence	Correctness and quality of the implementation, demonstration of a relevant use case, and evidence that the code supports the student's explanation.	10%
Research quality and use of sources	Appropriateness and reliability of sources, clear attribution, and evidence of independent understanding rather than uncritical reproduction.	10%
Teaching & presentation effectiveness	Clarity, organisation, use of visual aids, suitability for the intended audience, and effectiveness in communicating technical concepts.	10%
Response to questions	Ability to clarify explanations, justify technical claims, and reason about the structure beyond the prepared presentation.	10%

Deliverable Deadline:

Submission deadline for all materials (git repository, slide deck, any other materials to be used): 5th December

Presentations to be held in class across week 13.